04 Multimodal Large Model: Color Tracking

1. Brief Game Instructions

When the program starts, WonderEcho Pro announces "I am ready" and immediately begins monitoring the surrounding environment for sounds.

Say the wake-up word (the wake-up word depends on the firmware flashed, and by default, the wake-up word for our factory firmware is "Hello, HiWonder") to activate WonderEcho Pro. It will respond with "I am here."

You can control MentorPi's color tracking function using voice commands, such as "**Track the red blocks**." MentorPi converts the spoken command to text via the large model's ASR API, processes it through the large model, After processing, the large model's API system vocalizes the generated response, and the corresponding line-following program is executed.

2. Preparation

2.1 Verifying the WonderEcho Pro Firmware

By default, the wake-up word for the WonderEcho Pro factory firmware is "Hello, Hiwonder." If you wish to change the wake-up word to "MentorPi," please refer to the tutorial titled "20.1 Voice Module Overview and Installation\ 03 Firmware Flashing Method" for step-by-step instructions.

If you have previously flashed the device with different firmware—such as the version with the wake word "小幻小幻" or "mentorpi"—you must use the corresponding wake word to activate the device.

For example, if the "小幻小幻" firmware was installed, the correct wake word

would be “小幻小幻”; if the “mentorpi” firmware was installed, the wake word would be “mentorpi.”

In the following tutorial, we will use the default factory wake word “Hello, HiWonder” for demonstration purposes.

2.2 Obtaining and Configuring the Large Model API Key

By default, the program does not include the configuration of the Large AI Model-related API keys. Before activating features related to the Large AI Model, please refer to the section "02 Multimodal Large Model Applications > 01 Obtaining and Configuring Large Model API Keys" to configure the necessary keys. This step is mandatory and cannot be skipped, as it is crucial for the proper functioning and experience of the large model features.

2.3 Network Configuration

Note: The large language model used in this lesson is cloud-based. Before starting, please connect the robot to the internet using an Ethernet cable or switch it to local area network (LAN) mode.

The robot must be connected to the internet, either in STA (local network) mode or AP (direct connection) mode via Ethernet. For detailed instructions on network configuration, please refer to the course “20 Network Configuration Courses\ 01 Network Configuration Instructions”

3. Starting and Stopping the Game

Note:

① **The input commands must strictly observe case sensitivity and spacing.**

② **The robot must be connected to the internet, either in STA (local network) mode or AP (direct connection) mode via Ethernet.**

---

1) Start the robot and follow the instructions in “**8. Remote Tool Installation**

Connection → 8.4 Docker Container Introduction and Entry" to connect via the VNC remote desktop software. Then, switch to the ROS2 environment and select the appropriate language version as needed.

2) Power on the robot and remotely connect to the Raspberry Pi desktop via VNC.

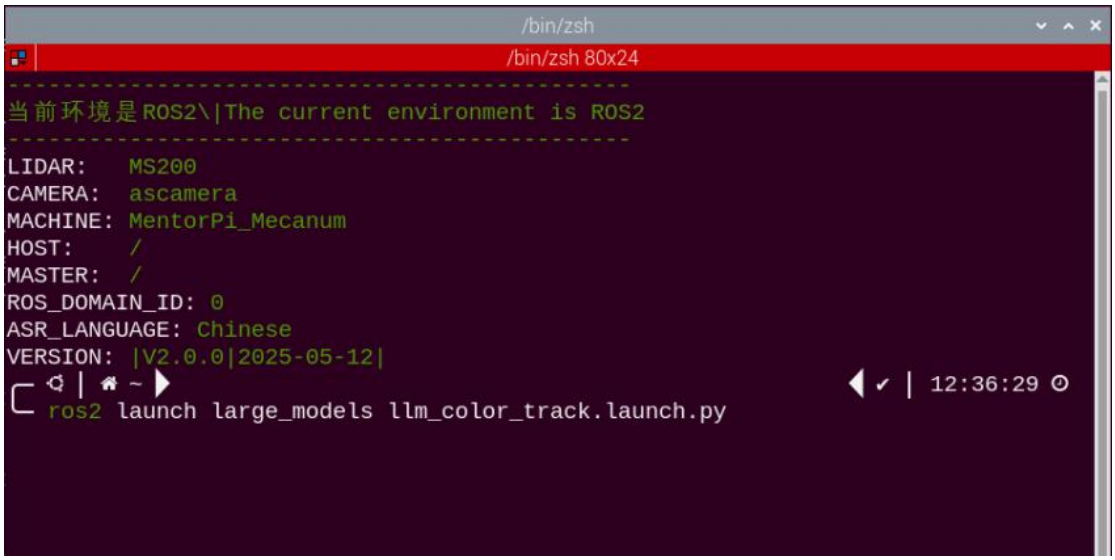
3) On the Raspberry Pi desktop, double-click  to open the command-line terminal and enter the ROS2 development environment.

4) Then, input the command to disable the auto-start service:

```
~/.stop_ros.sh
```

5) Then, input the command to start the game:

```
ros2 launch large_models llm_color_track.launch.py
```



6) When the command line shows the output below, initialization is complete. You can then say the wake-up phrase, "mentorpi."

7) When the following output appears in the command line, WonderEcho Pro will announce "I am Here," indicating that WonderEcho Pro has been successfully activated. At this point, the system will begin recording the user's command.

8) You can freely phrase your commands to control the movement of MentorPi, such as saying, "**Track the red blocks**."

```
[vocal_detect-10] [INFO] [1747364036.735027502] [vocal_detect]: asr...
[vocal_detect-10] [INFO] [1747364037.481808175] [vocal_detect]: Recording.....
[vocal_detect-10] [INFO] [1747364039.454928734] [vocal_detect]: Done recording
```

9) When the command line shows the output below, it indicates that the cloud-based large speech model's speech recognition service has successfully processed the user's command audio. The recognition result will be displayed under "publish_asr_result."

```
[vocal_detect-10] [INFO] [1747364041.505493300] [vocal_detect]: publish asr result:Track the red blocks.
[vocal_detect-10]
```

10) When the command line shows the output below, it indicates that the cloud-based large language model has successfully processed the user's command, thought through the instruction, and provided a verbal response ("response"), as well as designed an action that aligns with the user's command meaning.

At this stage, the robot sends a client request to the "Color Tracking" node to set the target color and initiate the tracking mode.

Note: The response is automatically generated by the large model, ensuring the accuracy of the meaning, though the wording and structure of the reply may vary.

```
[llm_color_track-13] [INFO] [1747364043.517668581] [function_call]: {"action": ["object_tracking('red')"], "response": "Spotting those red blocks like a hawk! Let's roll!"}
[llm_color_track-13] [INFO] [1747364043.519456852] [function_call]: msg:{"action": ["object_tracking('red')"], "response": "Spotting those red blocks like a hawk! Let's roll
!"}
```

11) When the following output appears in the terminal, it indicates that the Raspberry Pi has successfully called the cloud-based voice synthesis

service. WonderEcho Pro will then play the audio generated from the "response" in step 9.

Note: Once the Color Tracking mode is activated, the program will continue running this function. To stop it, press Ctrl+C in the terminal.

12) When the command line shows the output below, it indicates that one round of interaction has been completed. You can refer to Step 5) to reactivate the voice device using the wake-up word and begin a new round of interaction.

13) To exit this feature, simply press “Ctrl+C” in the terminal. If it doesn’t close on the first attempt, you may need to press it multiple times.

4. How It Works

After the mode is activated, you can freely give voice commands such as “Track the red blocks” to control MentorPi’s autonomous tracking. In this mode, MentorPi will follow the red color blocks.

If the red blocks are not detected for more than 5 seconds or the system is reawakened, the color tracking task will exit and return to the wake-up state.

For the rules governing text responses and action design by the large model, please refer to the prompt settings within the program. Detailed instructions can be found in the PROMPT section of /home/ubuntu/ros2_ws/src/large_models/large_models/function_call.py

The actions available in this mode are predefined within the program library and are limited in number. MentorPi’s movements are controlled by calling functions that correspond to action strings designed by the large model. If a command involves

an action that is not predefined, the program will be unable to execute it; however, the large model will still recognize the command and provide a response.

5. Brief Program Analysis

The launch file for this program is located at:

The launch file is saved in: **/home/ubuntu/ros2_ws/src/large_models/launch/llm_color_track.launch.py**

```
def launch_setup(context):
    mode = LaunchConfiguration('mode', default=1)
    mode_arg = DeclareLaunchArgument('mode', default_value=mode)

    app_package_path = get_package_share_directory('app')
    large_models_package_path = get_package_share_directory('large_models')

    object_tracking_node_launch = IncludeLaunchDescription(
        PythonLaunchDescriptionSource(
            os.path.join(app_package_path, 'launch/object_tracking_node.launch.py
')),
        launch_arguments={
            'debug': 'true',
        }.items(),
    )

    large_models_launch = IncludeLaunchDescription(
        PythonLaunchDescriptionSource(
            os.path.join(large_models_package_path, 'launch/start.launch.py')),
        launch_arguments={'mode': mode}.items(),
    )

    llm_color_track_node = Node(
        package='large_models',
        executable='llm_color_track',
        output='screen',
    )
```

- **Color Tracking Control**

The object_tracking_node is responsible for managing object color tracking.

- **Large Model Control Node**

The start command launches the large model control node, enabling functions such as speech detection, semantic understanding, task execution, and response playback.

Response Playback

5.1 Main Control Program Analysis

The source code is located at:

/home/ubuntu/ros2_ws/src/large_models/large_models/llm_color_track.py

```
import re
import time
import math
import rclpy
import ast
import threading
from speech import speech
from rclpy.node import Node
from large_models.config import *
from geometry_msgs.msg import Twist

from std_msgs.msg import String, Bool
from std_srvs.srv import Trigger, SetBool, Empty
from rclpy.executors import MultiThreadedExecutor
from rclpy.callback_groups import ReentrantCallbackGroup

from large_models_msgs.srv import SetModel,SetString
from ros_robot_controller_msgs.msg import RGBState,RGBStates, SetPWMServoState, PWMServoState
```

- **Library Files Import**

The program imports the following modules and libraries to enable functionalities such as ROS2 node creation, message publishing, service calls, threading, and audio playback:

- **Creates and initializes the ROS2 node**

It initializes the ROS2 environment by calling rclpy.init() and creates a node instance by inheriting from the Node class.

```
class FunctionCall(Node):
    def __init__(self, name):
        rclpy.init()
        super().__init__(name)
```

- **Initializing Variables**

Define the following variables:

action: list of actions

llm_result: string containing the response from the large language model

running: flag indicating the running status

result: parsed result data

```
59        self.action = []
60        self.interrupt = False
61        self.llm_result = ''
62        self.running = True
63        self.result = ''
```

◆ **Publisher and Subscriber Initialization**

① tts_text_pub: This publisher publishes text messages to the tts_node/tts_text topic. Sending text to the speech synthesis module: This allows the robot to provide voice feedback to the user.

② mecanum_pub： This publisher sends Twist messages to the /controller/cmd_vel topic. The Twist message includes the robot's linear velocities (x, y) and angular velocity (z) to control its movement.

③ llm_result_callback: Subscribes to the agent_process/result topic, which contains commands generated by the Large Language Model (LLM).

④ play_audio_finish_callback: Subscribes to the tts_node/play_finish topic, which notifies the program when voice playback is complete.

```
67        self.mecanum_pub = self.create_publisher(Twist, '/controller/cmd_vel', 1)
68
69        timer_cb_group = ReentrantCallbackGroup()
70        self.tts_text_pub = self.create_publisher(String, 'tts_node/tts_text', 1)
71        self.awake_client = self.create_client(SetBool, 'vocal_detect/enable_wake
    up')
72        self.create_subscription(String, 'agent_process/result', self.llm_result_
    callback, 1)
73        self.create_subscription(Bool, 'tts_node/play_finish', self.play_audio_fi
    nish_callback, 1, callback_group=timer_cb_group)
74
75        self.awake_client = self.create_client(SetBool, 'vocal_detect/enable_wake
    up')
76        self.awake_client.wait_for_service()
77        self.create_subscription(Bool, 'vocal_detect/wakeup', self.wakeup_callbac
    k, 1)
78        self.set_model_client = self.create_client(SetModel, 'agent_process/set_m
    odel')
79        self.set_model_client.wait_for_service()
80
81        self.set_prompt_client = self.create_client(SetString, 'agent_process/set
    _prompt')
82        self.set_prompt_client.wait_for_service()
83
```

◆ **Determine whether the input is in Chinese**

```
25 if os.environ["ASR_LANGUAGE"] == 'Chinese':
26     PROMPT = '''
```

◆ **Send Prompt**

The PROMPT variable defines the prompt sent to the LLM. The LLM parses user input and generates JSON-formatted output containing actions and responses.

```
51     PROMPT = '''
52 # Role
53 You are an intelligent visual omni-directional wheeled robot that needs to generate corresponding JSON commands based on the input content.
54
55 ## Requirements and Limitations
56 1. Based on the input action content, find the corresponding command in the action function library and output the corresponding command.
57 2. Craft a concise (10 to 30 words), witty, and ever-changing feedback message to make the interaction lively and interesting.
58 3. Directly output the JSON result without analysis or any additional content.
59 4. Format: {'action': ['xx', 'xx'], 'response': 'xx'}
60
61 ## Structure Requirements:
62 - The `"action"` key should contain an array of function name strings in the order of execution. If no corresponding action function is found, the action should output a
   n empty array `[]`.
63 - The `"response"` key should be paired with a carefully crafted brief reply that fits the above word count and style requirements.
64
65 ## Action Function Library
66 - Track a red object: object_tracking('red')
67
68 ### Task Examples:
69 Input: Track a red object
70 Output: {"action": ["object_tracking('red')"], "response": "Locking onto the red target, tracking with ease!"}
71 Input: Track the color of leaves
72 Output: {"action": ["object_tracking('green')"], "response": "Locking onto the green target, tracking with ease!"}
73 Input: Track the color of the ocean
74 Output: {"action": ["object_tracking('blue')"], "response": "Locking onto the blue target, tracking with ease!"}
75 '''
```

◆ **init_process Method**

① Sets the LLM prompt (PROMPT) and stops the mecanum wheeled robot. Play the Startup Audio. Launches a new thread to run the process method. Creates a ROS 2 service ~/init_finish to signal node initialization completion.

② Iterates through the action list, invoking corresponding ROS 2 services based on each action type.

③ For the "line_following" action, extracts color information and calls the related service to start line tracking.

④ After completing all actions, resets the interrupt flag, clears the llm_result variable, and re-enables voice wake-up.

```
107     def init_process(self):
108         self.timer.cancel()
109         msg = SetString.Request()
110         msg.data = PROMPT
111         self.set_prompt_client.call_async(msg)
112         #self.send_request(self.set_prompt_client, msg)
113         self.mecanum_pub.publish(Twist())
114         speech.play_audio(start_audio_path)
115 
116         threading.Thread(target=self.process, daemon=True).start()
117 
118         self.create_service(Trigger, '~/init_finish', self.get_node_state)
119         self.get_logger().info('\033[1;32m%s\033[0m' % 'start')
120 
```

- **Main Function**

Main Function Execution

```
214 def main():
215     node = FunctionCall('function_call')
216     try:
217         rclpy.spin(node)
218     except KeyboardInterrupt:
219         print('shutdown')
220     finally:
221         rclpy.shutdown()
222 
223 if __name__ == "__main__":
224     main()
```

5.2 Large Model Line-Following Logic Analysis

- **Large Model Interaction Logic Analysis**

The configuration file of this program is saved in:

/home/ubuntu/ros2_ws/src/app/app/object_tracking.py

Function: Used to receive target color information from the large model. The large model sends the target color name to this node by calling the ~/set_large_model_target_color service.

```
    def set_large_model_target_color_srv_callback(self, request, response):
        self.get_logger().info('\033[1;32m%s\033[0m' % "set_large_model_target_color")
        with self.lock:
            color_name = request.data
            self.get_logger().info(f"请求的颜色名称：{color_name}")

            self.tracker = None  # Reset the tracker
            self.large_model_tracking = True

            try:
                # 确定使用哪个相机类型：如果当前相机类型是 'ascamera'，则使用 'Stereo'，否则使用 'Mono'
                camera_type = 'Stereo' if self.camera_type == 'ascamera' else 'Mono'
                # 将相机类型传递给 ObjectTracker 类
                self.tracker = ObjectTracker(None, self, color_name, True)
                try:
                    temp = self.tracker.lab_data['lab'][camera_type][color_name]
                    self.get_logger().info(f"成功读取 颜色配置：self.lab_data['lab'][{camera_type}][{color_name}]")
                except KeyError as e:
                    self.get_logger().error(f"读取 颜色配置失败：{e}")
                    raise
                self.mecanum_pub.publish(Twist())
                response.success = True
                response.message = "set_large_model_target_color"
            except Exception as e:
                response.success = False
                response.message = str(e)
                self.get_logger().error(f"设置目标颜色时发生错误：{e}")
            return response
```

◆ **Image_callback Callback Function**

① If the tracker exists, its __call__ method is invoked to process the image and publish motion control commands based on the results.

② If is_running is True, the motion control commands are published. Otherwise, the output of the PID controller is cleared.

```
    def image_callback(self, ros_image):
        # 将ros格式(rgb)转为opencv的rgb格式(convert RGB format of ROS to that of OpenCV)
        cv_image = self.bridge.imgmsg_to_cv2(ros_image, "rgb8")
        rgb_image = np.array(cv_image, dtype=np.uint8)
        self.image_height, self.image_width = rgb_image.shape[:2]

        result_image = np.copy(rgb_image)  # 显示结果用的画面(the image used for display the result)
        with self.lock:
            # 颜色拾取器和识别追踪互斥，如果拾取器存在
                        # 颜色拾取器和识别追踪互斥，如果拾取器存在就开始拾取(color picker and object tracking are mutually exclusive. If the color picker exists, start picking colors)
            if self.color_picker is not None:  # 拾取器存在(color pick exists)
                target_color, result_image = self.color_picker(rgb_image, result_image)
                if target_color is not None:
                    self.color_picker = None
                    self.tracker = ObjectTracker(target_color, self)

                    self.get_logger().info("target color: {}".format(target_color))
            else:
                if self.tracker is not None:
                    try:
                        result_image, twist = self.tracker(rgb_image, result_image, self.threshold)

```

◆ **Automatic Stop Mechanism**

① The start_stop_timer method creates a timer that calls the stop_after_lose method after 5 seconds.

② The stop_after_lose method stops the robot's movement and sets the is_running flag to False, thereby stopping the line-following process.

```
390     def start_stop_timer(self):
391         if self.target_lost_timer is None:
392             self.target_lost_timer = threading.Timer(5.0, self.stop_after_lose)
393             self.target_lost_timer.start()
394             self.target_lost = True
395 
396     def stop_after_lose(self):
397         """Stops the robot after the target is lost for a specified duration."""
398         with self.lock:
399             if self.large_model_tracking:
400                 self.get_logger().warn("目标丢失超过5秒，停止移动 (Target lost for more than 5 seconds, stopping movement)")
401                 self.mecanum_pub.publish(Twist())
402                 self.is_running = False
403                 self.target_lost = False
404                 self.target_lost_timer = None
```

◆ **Wake-up Stop**

Upon detecting a voice wake-up signal, the robot will stop only if large_model_tracking is set to True. This ensures that the stop command triggered by the voice wake-up signal is executed solely when color tracking is active under the large model's control.

```
449     def wakeup_callback(self, msg):
450         """Callback function for the /vocal_detect/wakeup topic."""
451         if msg.data:
452             self.get_logger().info("Wake-up detected, exiting large model tracking.")
453             with self.lock:
454                 if self.large_model_tracking:
455                     self.large_model_tracking = False
456                     self.is_running = False
457                     self.mecanum_pub.publish(Twist())  # Stop the robot
458                     if self.target_lost_timer is not None:
459                         self.target_lost_timer.cancel()
460                         self.target_lost_timer = None
461                     self.tracker = None  # 清除tracker
462                     self.get_logger().info("Large model tracking stopped.")
```